FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery

Part II — Runtime Invariant Algebra

Sizhe Tan
with chatGPT (OpenAI)
2026-07-15

Repository: https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA

Abstract

Transformer Attention is widely recognized as the central mechanism underlying modern Large Language Models (LLMs).
Traditionally, Attention is interpreted as a differentiable weighting mechanism that computes contextual dependencies among tokens.
Within the Runtime Invariant Algebra proposed by FTRIA, Attention admits a complementary structural interpretation.
Rather than viewing Attention merely as weighted aggregation,
this paper interprets Attention as a process of **Dynamic Runtime Invariant Discovery**.
During inference,
multiple candidate Runtime Invariants are continuously evaluated, assembled,
and refined according to the current runtime context.
Attention therefore functions as a dynamic Runtime Invariant Discovery Operator operating inside an approximate Runtime Invariant Configuration Space.
This interpretation naturally connects Transformer computation with the Runtime Invariant framework established throughout FTRIA.

1. Existing Algorithm

Transformer models compute contextual representations through the Attention mechanism.
Conceptually,
Query

Key

Value

↓

Attention

↓

Contextual Representation

Attention dynamically determines which contextual information contributes most strongly to the next computation.
This mechanism enables Transformers to adapt continuously to changing runtime contexts.

2. Traditional Interpretation

Attention is commonly interpreted as

- contextual weighting,
- relevance estimation,
- similarity computation,
- differentiable memory access,
- information aggregation.

Each token evaluates relationships with surrounding tokens,
producing context-dependent representations.
This interpretation explains the computational behavior of Transformer models,
but leaves open an important structural question:
What stable structure is Attention attempting to assemble?

3. Runtime Invariant Interpretation

Within Runtime Invariant Algebra,
Attention may be interpreted as **Dynamic Runtime Invariant Discovery**.
Conceptually,
Candidate Runtime Invariants

↓

Context Evaluation

↓

Dynamic Runtime Invariant Discovery

↓

Current Runtime Invariant

Rather than selecting isolated words,
Attention continuously searches for the Runtime Invariant most consistent with the current execution context.
The discovered Runtime Invariant guides subsequent computation.

4. Dynamic Discovery Rather Than Static Encoding

Embeddings provide relatively stable Runtime Invariant approximations.
Attention performs a different task.
It dynamically reorganizes,
weights,
and combines those approximations according to the current context.
Consequently,
embedding and Attention perform complementary Runtime functions.
Embedding

↓

Approximate Runtime Invariant Encoding

↓

Attention

↓

Dynamic Runtime Invariant Discovery

Encoding provides candidates.
Attention discovers the most appropriate Runtime Invariant for the current runtime state.

5. Runtime Context as Discovery Constraint

Attention does not operate in isolation.
Its discovery process is constrained by

- surrounding tokens,
- previous Runtime states,
- learned representations,
- structural consistency,
- prediction objectives.

These constraints continuously reshape the Runtime Invariant Discovery process.
Consequently,
different contexts may activate different Runtime Invariants despite sharing many common embedding components.

6. Relationship to Common Concept Core

Common Concept Core (CCC) explicitly discovers Runtime Invariants from multiple observations.
Attention performs a related operation,
but dynamically during runtime.
Conceptually,
CCC

↓

Explicit Runtime Invariant Discovery

Attention

↓

Dynamic Runtime Invariant Discovery

The computational mechanisms differ,
yet both seek stable structural interpretations from multiple candidates.
CCC performs explicit structural reasoning.
Attention performs large-scale statistical approximation.

7. Runtime Invariant Discovery During Inference

Inference may therefore be interpreted as a continuous Runtime Invariant

Discovery process.
Each newly generated token updates the runtime context.
The updated context changes the candidate Runtime Invariants.
Attention then performs another discovery step.
Conceptually,
Runtime Context

↓

Attention

↓

Runtime Invariant Discovery

↓

Prediction

↓

Updated Runtime Context

↓

Repeat

Transformer inference thus becomes an iterative Runtime Invariant Discovery process.

8. Engineering Implications

Viewing Attention as Dynamic Runtime Invariant Discovery provides several practical insights.
It helps explain

- contextual reasoning,
- long-range dependency modeling,
- multi-hop reasoning,
- in-context learning,
- retrieval-augmented generation,
- adaptive representation construction.

It also suggests future research directions,
including

- explicit Runtime Invariant reasoning,
- hybrid symbolic–neural discovery,
- Runtime Invariant debugging,
- interpretable Attention,
- Runtime-aware Transformer architectures.

Rather than replacing Attention,
Runtime Invariant Algebra provides a structural interpretation of its runtime behavior.

Key Takeaways

- Transformer Attention may be interpreted as Dynamic Runtime Invariant Discovery.
- Embeddings provide approximate Runtime Invariant encodings, while Attention dynamically discovers the most appropriate Runtime Invariant for the current context.
- Runtime context continuously constrains the discovery process.
- Transformer inference can be viewed as iterative Runtime Invariant Discovery.
- Common Concept Core and Attention perform analogous Runtime functions using different computational mechanisms.
- Runtime Invariant Discovery Algebra provides a unified structural interpretation of symbolic and neural discovery processes.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra
- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding

Trigger Algebra

- FTRIA-120 — Runtime Invariant Trigger Algebra
- FTRIA-125 — Transformer Attention as Runtime Triggering

Foundations

- FTRIA-001 — Runtime Invariant Degrees of Freedom
- FTRIA-007 — Runtime-Invariant Configuration Space